CCY2001: Introduction to Cybersecurity

Lecture 5 – Introduction to Cryptography

Dr. Hany Elshall

CCIT

March 9, 2026

Lecture Outline

What is Cryptography?

What is Cryptography?

Definition

Cryptography is the science of using mathematical algorithms and coded systems to *secure communication* by protecting information from unauthorised access.

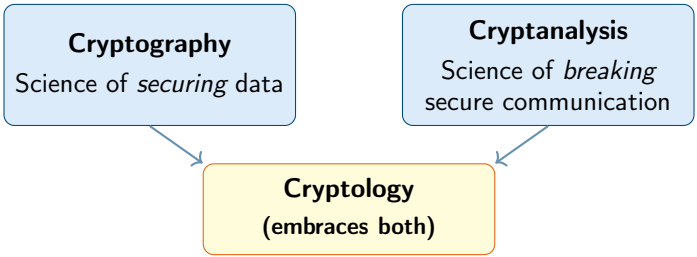
In simple words...

- Hidden / disguised / encrypted communications
- Turns readable data into unreadable form
- Only authorised parties can reverse the process

Real-life analogy

Sending a letter in a locked box. Only the person with the right key can open it and read the message.

Cryptography vs. Cryptanalysis vs. Cryptology



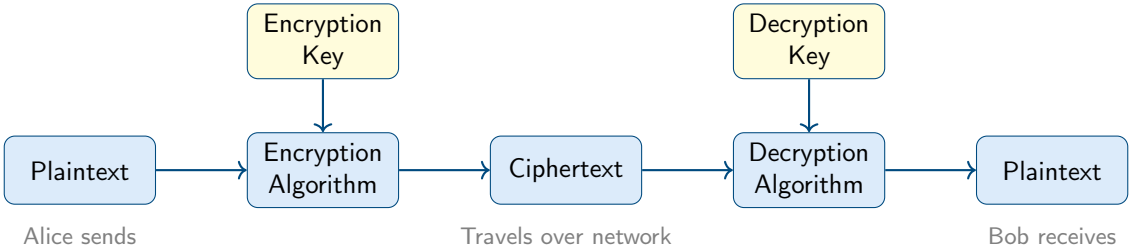
A **cryptanalyst** (attacker) tries to decode encrypted data *without* having the encryption key, using analytical reasoning, mathematics, pattern finding, and sometimes luck.

Core Terminology

Core Terminology

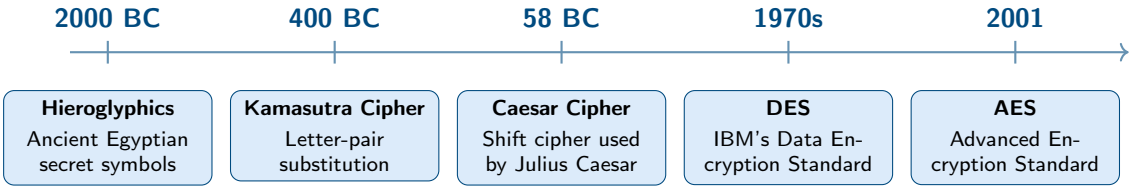
Term	Definition
Plaintext	Original, readable message (also called *cleartext*)
Ciphertext	Scrambled, unreadable form of the message
Encryption	Converting plaintext $\rightarrow$ ciphertext
Decryption	Converting ciphertext $\rightarrow$ plaintext
Cipher	Algorithm used for encryption / decryption
Key	Parameter controlling the cipher's output
Cryptosystem	Complete implementation: algorithms + infrastructure

The Cryptosystem Model



History of Cryptography

A Brief History



Hieroglyphics – The First Known Cryptography

- Oldest known use of cryptography: ≈**4000 years ago**
- Ancient Egyptians communicated using **hieroglyphs**
- Some inscriptions used non-standard or unusual symbols — a deliberate attempt to obscure meaning
- Roots of cryptography found in **Roman & Egyptian civilisations**

"As civilisations evolved, the need to communicate secretly drove the continuous evolution of cryptography."

Ancient Egyptian Symbols

Kamasutra Cipher (≈400 BC)

Kamasutra Cipher

A substitution cipher from ancient India. Letters of the alphabet are **randomly paired**; each letter is replaced by its partner.

Key: A permutation (random pairing) of the alphabet

- Originally described in the *Kama Sutra* by Vātsyāyana
- Purpose: teaching women to hide secret messages
- The key is the pairing arrangement

Example pairing (partial)

Plain	↔	Cipher
I	↔	D
N	↔	W
T	↔	R
E	↔	C
R	↔	T
S	↔	F

`INTERNET` → `DWRCTWCR`

`SOCIETY` → `FKEDCRL`

Caesar Shift Cipher

Caesar Cipher

Shift every letter in the message by a fixed number of positions in the alphabet. Julius Caesar used a **shift of 3**.

How it works (shift = 3):

$$A \to D, \quad B \to E, \quad Z \to C$$

- **Encrypt:** Plaintext $\xrightarrow{+3}$ Ciphertext
- **Decrypt:** Ciphertext $\xrightarrow{-3}$ Plaintext
- Used to protect *military messages*

Example (shift

Plaintext: HELLO
Ciphertext: KHOOR

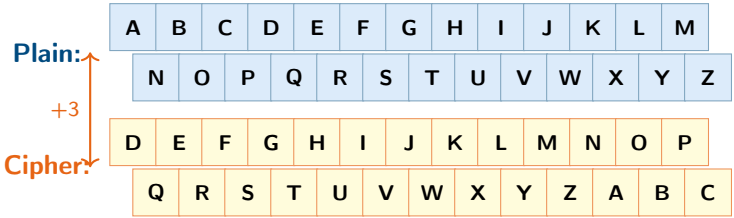
H → K
L → O
O → R

E → H
L → O

Weakness

Only 25 possible keys – easy to crack by brute force!

Caesar Cipher – Alphabet Shift Visualisation



Encrypt "ATTACK" with shift 3

A T T A C K ⟶ D W W D F N

Goals & Services of Cryptography

Goals & Services of Cryptography

Primary Goal: Secure data *at rest* and *in transit* across insecure networks.

- **Confidentiality (Secrecy)**

  Only the **intended receiver** can read the message – data stays secret even if intercepted.

- **Integrity (Anti-tampering)**

  Assures the receiver that the message has **not been altered** in transit.

- **Authentication**

  Cryptography helps establish and **prove identity** – who you are communicating with.

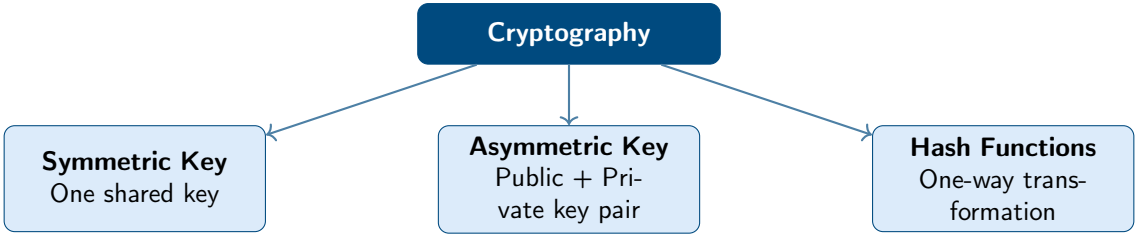
- **Non-repudiation**

  A mechanism to **prove the sender really sent** the message – they cannot deny it later.

These four services map directly to the CIA+ model you learned in earlier weeks of this course.

Types of Cryptography

Three Main Types of Cryptography

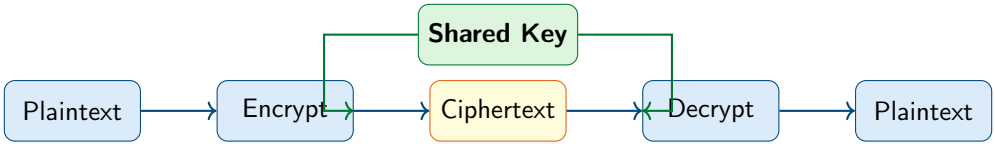


Symmetric Key Cryptography

Symmetric Key Cryptography – Overview

Symmetric Key Cryptography

Also called **Secret Key** or **Conventional** Cryptography.
The **same key** is used for both encryption and decryption.



✓ **Advantages**

- Fast and computationally efficient
- Suitable for large volumes of data

× **Drawbacks**

- Key distribution problem
- Scalability issues in large networks
- Compromised key exposes all data

Symmetric Encryption – Simple Example (Shift = 5)

Scenario: Alice sends `"HELLO"` to Bob using a Caesar-style cipher with **key = 5**

Alice – Encryption:

Plain	$\xrightarrow{+5}$	Cipher
H	→	M
E	→	J
L	→	Q
L	→	Q
O	→	T
`HELLO`		`MJQQT`

Bob – Decryption:

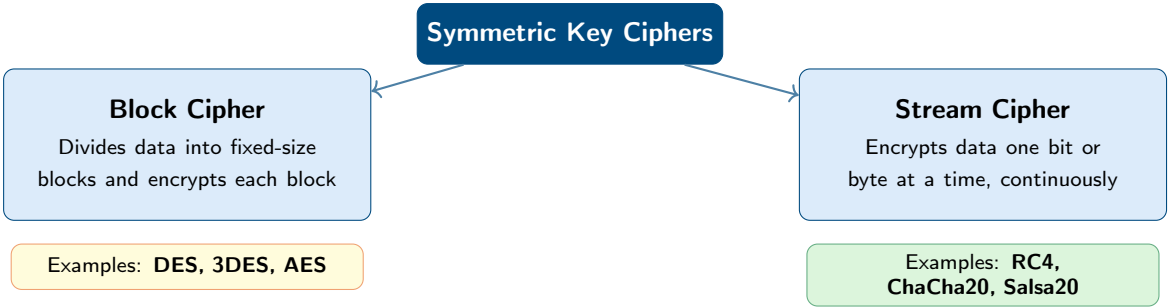
Cipher	$\xrightarrow{-5}$	Plain
M	→	H
J	→	E
Q	→	L
Q	→	L
T	→	O
`MJQQT`		`HELLO`

Both parties **must already know** the key (= 5). This is the key-distribution problem.

Symmetric Ciphers: Two Categories

Two Types of Symmetric Ciphers

Symmetric key algorithms can be divided into two fundamental categories based on **how** they process plaintext data.



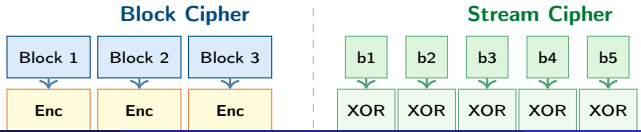
Block Cipher vs. Stream Cipher

Block Cipher

- Splits plaintext into **fixed-size blocks** (e.g. 64 or 128 bits per block)
- Encrypts the **entire block at once**
- If last block is too short $\Rightarrow$ **padding** is added
- Uses **multiple rounds** of substitution & permutation
- Generally **slower** but highly secure
- Best for: files, databases, disk encryption

Stream Cipher

- Encrypts data **one bit or byte at a time**
- Generates a **keystream** from the key and XORs it with plaintext
- **No padding** needed – works on any length
- Faster and **lower memory usage**
- Best for: real-time audio/video, network streams, mobile



Stream Cipher Examples

RC4 (Rivest Cipher 4)

- Designed by **Ron Rivest** (RSA) in **1987**
- Variable key length: 40 – 2048 bits
- Byte-oriented stream cipher
- Was widely used in: **WEP** (Wi-Fi), **WPA**, **SSL/TLS**
- Simple and very **fast** in software

RC4 Status

Deprecated. Multiple vulnerabilities found (biased keystream bytes). Prohibited in TLS since RFC 7465 (2015). **Do not use in new systems.**

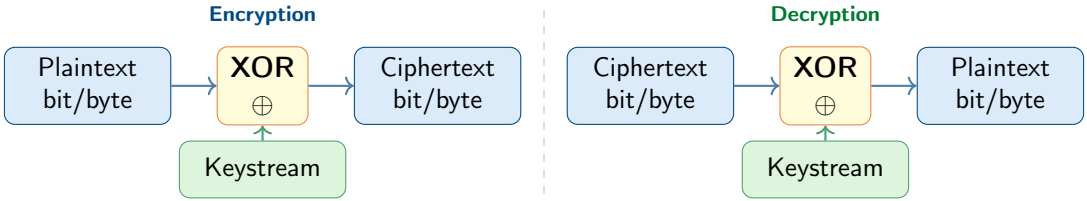
ChaCha20

- Designed by **Daniel J. Bernstein** in **2008**
- Fixed key: **256-bit**; Nonce: **96-bit**
- Based on *add-rotate-XOR* (ARX) operations
- **Modern and highly secure** – no known practical attack
- Used in: **TLS 1.3**, **QUIC**, Android, OpenSSH
- Faster than AES on devices *without* hardware acceleration

Stream Cipher – How the XOR Operation Works

The Core Idea

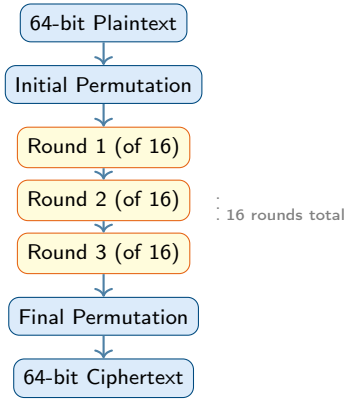
A stream cipher generates a **pseudorandom keystream** from the secret key. Each plaintext bit/byte is XOR-ed with the corresponding keystream bit/byte to produce ciphertext.



Key insight: XOR is its own inverse! Same operation for both encryption and decryption.
$P \oplus K = C$ (encrypt) $C \oplus K = P$ (decrypt)

Data Encryption Standard (DES)

- Developed by **IBM**, endorsed by NSA (1974–2002)
- Was the go-to algorithm for ≈30 years
- **Symmetric block cipher**
- Block size: **64 bits**
- Key size: **56 bits** (effective)
- Uses **16 rounds** of Feistel structure
- Includes substitution (S-boxes) and permutation



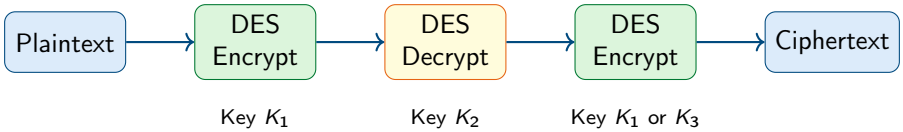
Why DES is deprecated

56-bit key $= 2^{56} \approx 72$ trillion combinations.
Modern hardware can crack DES in **hours**.

Triple DES (3DES)

Triple DES

An enhanced version of DES that applies the DES algorithm **three times** in an Encrypt–Decrypt–Encrypt (EDE) sequence.



- Effective key: **112-bit** (2-key) or **168-bit** (3-key)
- Block size still 64-bit
- More secure than DES, but **slower**

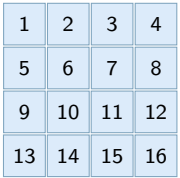
Status

NIST deprecated 3DES in 2023.
Being phased out in favour of **AES**.

Advanced Encryption Standard (AES)

AES at a Glance

Adopted by:	U.S. Government (2001)
Block size:	128 bits
Key sizes:	128 / 192 / 256 bits
Rounds:	10 / 12 / 14
Structure:	Substitution-Permutation
Status:	Current world standard



AES 4×4 State Matrix

Each cell = 1 byte; total = 128 bits

- Replaced DES as the global standard
- Extensively analysed – no practical attack known
- Used in: Wi-Fi (WPA2), TLS/HTTPS, disk encryption...

AES operations per round:
SubBytes → ShiftRows → MixColumns → AddRoundKey

Symmetric Algorithms – Comparison

Property	DES	3DES	AES
Year introduced	1977	1999	2001
Block size	64-bit	64-bit	128-bit
Key size	56-bit	112/168-bit	128/192/256-bit
Rounds	16	48	10/12/14
Speed	Fast	Slow	Fast
Security	✗ Broken	Weak	✓ Strong
Status	Deprecated	Deprecated	Current standard

Drawbacks of Symmetric Key Cryptography

The Key Distribution Problem

Both parties must share the secret key **before** communicating.
How do you securely exchange the key over an insecure network?

1. **Key Management Nightmare**
   In a network of n users, you need $\frac{n(n-1)}{2}$ unique keys!
   Example: 1000 users $\Rightarrow$ 499 500 keys
2. **Key Compromise = Total Exposure**
   If the shared key is stolen, every past and future message is exposed.
3. **No Non-repudiation**
   Since both parties share the same key, either could have created any message.

These problems motivated the invention of **Public Key (Asymmetric) Cryptography**.

Asymmetric Key Cryptography

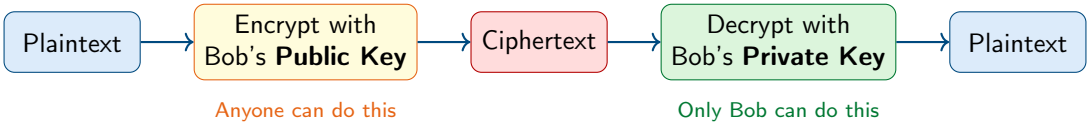
Asymmetric Key Cryptography – Overview

Asymmetric (Public Key) Cryptography

Uses **two mathematically linked keys**:

- **Public Key** – shared openly with everyone; used to *encrypt*
- **Private Key** – kept secret by the owner; used to *decrypt*

What one key encrypts, **only the other can decrypt**.



Symmetric vs. Asymmetric Encryption

Feature	Symmetric	Asymmetric
Keys used	1 shared key	2 keys (public + private)
Key exchange	Hard (insecure channel)	Easy (public key is open)
Speed	Fast	Slow
Use case	Bulk data encryption	Key exchange, signatures
Scalability	Poor (n^2 keys)	Good ($2n$ keys)
Non-repudiation	No	Yes (with private key)
Examples	AES, DES	RSA, ECC

In practice, systems use **hybrid encryption**: asymmetric to exchange a session key, then symmetric to encrypt the bulk data.
Example: TLS/HTTPS uses RSA/ECC to share an AES key.

RSA Algorithm

RSA

Published in **1977** by Rivest, Shamir, and Adleman.
First algorithm suitable for both **encryption** and **digital signing**.

Mathematical foundation:

- Easy to multiply two large primes $p \times q = n$
- Extremely hard to *factorise* n back into p and q
- This asymmetry provides security

Key sizes: 2048-bit minimum (4096-bit recommended today)

The RSA Challenge

In **1991**, RSA Laboratories released 54 semi-prime numbers and offered prizes for factorisation.

Competition ended **2007**: only **12 out of 54** factors found!

Results shaped recommendations for minimum RSA key sizes.

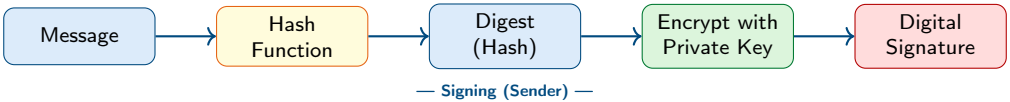
Digital Signatures

Digital Signature

A cryptographic mechanism to verify:

- **Authenticity** – the message really came from the claimed sender
- **Integrity** – the message was not changed in transit
- **Non-repudiation** – the sender cannot deny sending it

Based on **asymmetric cryptography**.



Digital Signature Standard (DSS)

- U.S. Federal standard **FIPS 186**, published by NIST in **1994**
- Specifies algorithms for generating and verifying digital signatures
- Became the U.S. government standard for *authentication* of electronic documents

Allowed algorithms under DSS:

- **DSA** – original Digital Signature Algorithm
- **RSA** – added in later revisions
- **ECDSA** – Elliptic Curve DSA (modern, efficient)

Core DSS Components

1. **Approved Algorithms:** DSA, RSA, ECDSA
2. **SHA:** Hash function produces a "digital fingerprint"
3. **PKI:** Manages public/private key pairs

DSS does not dictate *how* to sign – it specifies *which* algorithms are approved.

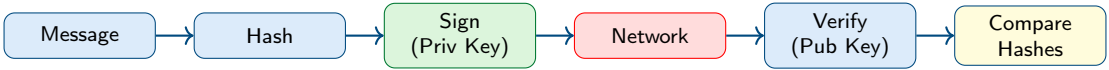
How DSS Works (RSA-Based Signatures)

Step 1 – Signing (Sender)

1. Write the message
2. Compute **hash** of the message (e.g. SHA-256)
3. **Sign** the hash with the sender's **private key**
4. This encrypted hash = **digital signature**
5. Send: message + signature

Step 2 – Verification (Receiver)

1. Receive: message + signature
2. **Recompute** the hash of the received message
3. **Verify** the signature using sender's **public key**
4. Compare both hashes
5. If they match ⇒ **valid signature**

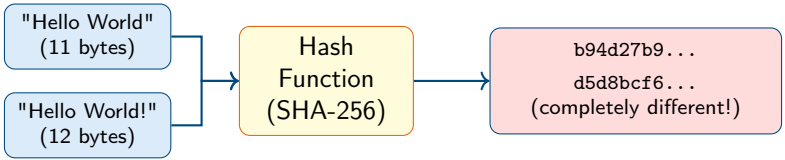


Hash Functions

Hash Functions – Overview

Hash Function

Takes **any input** (text, file, message) and produces a **fixed-size output** called a *hash* or *digest*.

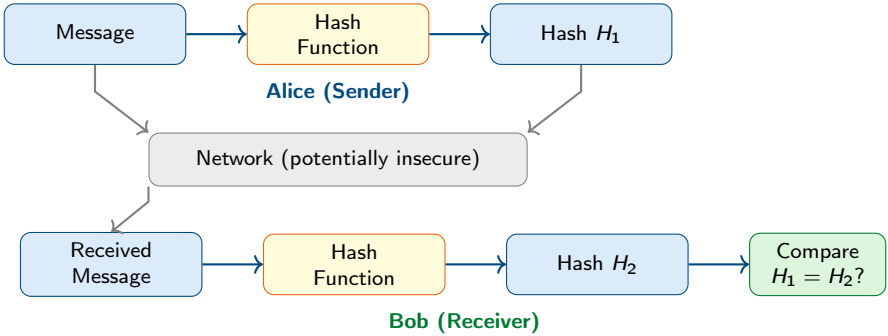


Hash Functions – Properties

Key properties:

1. **Fixed output size** regardless of input size
   (SHA-256 always outputs 256 bits)
2. **One-way (irreversible)** – you cannot reverse a hash
3. **Deterministic** – same input always gives same hash
4. **Avalanche effect** – tiny change in input $\Rightarrow$ completely different hash
5. **Collision resistant** – infeasible to find two inputs with the same hash

How Hash Functions Ensure Data Integrity



MD5 – And the Collision Problem

- **MD5**: 128-bit hash, output = 32 hex characters
- Once widely used for passwords and file integrity
- In **2005**: Wang & Yu (Shandong University) demonstrated a *collision* – two different files with the **same MD5 hash**

Collision Attack

A **collision** means two different inputs produce the same hash.
This breaks the "collision resistant" property!
An attacker could substitute a malicious file that has the same hash as the original.

MD5 Collision (2005)

`file1.dat` and `file2.dat` are **different files** yet both produce:

`MD5 = a4c0d35c...`

Checking:

```
$ md5sum file1.dat
a4c0d35c...
$ md5sum file2.dat
a4c0d35c...
```

Same hash – different files!

SHA – Secure Hash Algorithm Family

- Designed by **NSA**, published by **NIST**
- A family of hash functions: SHA-1, SHA-2, SHA-3
- Because of attacks on MD5 and SHA-1, NIST sought a new alternative
- In **October 2012**: NIST chose the **Keccak** algorithm as the new **SHA-3** standard

Algorithm	Output	Status
MD5	128-bit	Broken
SHA-1	160-bit	Broken
SHA-256	256-bit	Secure
SHA-512	512-bit	Secure
SHA-3	Variable	Current

Rule of thumb: Use **SHA-256** or **SHA-3** for any new security-sensitive application. Never use MD5 or SHA-1 for security purposes.

Summary & Review

Key Takeaways

- **Concepts**
  - Plaintext, Ciphertext, Cipher, Key
  - Encryption / Decryption
  - Cryptanalysis vs. Cryptography
- **Algorithms**
  - Symmetric: DES $\rightarrow$ 3DES $\rightarrow$ **AES**
  - Asymmetric: **RSA**, Digital Signatures, DSS
  - Hash: MD5 (broken) $\rightarrow$ **SHA-256/SHA-3**
- **History**
  - Hieroglyphs ($\approx$2000 BC)
  - Kamasutra Cipher (400 BC)
  - Caesar Cipher (shift of 3)
- **Services**
  - Confidentiality, Integrity
  - Authentication, Non-repudiation

Review Questions

1. What is the difference between **cryptography** and **cryptanalysis**?
2. Describe the **key distribution problem** in symmetric cryptography. How does asymmetric cryptography solve it?
3. Compare **DES**, **3DES**, and **AES** in terms of key size and security.
4. Explain how a **digital signature** provides non-repudiation.
5. What is a **hash collision**? Why does the MD5 collision discovered in 2005 matter for security?
6. Why is it common to use **hybrid encryption** (symmetric + asymmetric) in real-world systems like HTTPS?

Further Reading & References

- Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.
- Paar, C., & Pelzl, J. (2010). *Understanding Cryptography*. Springer.
- NIST. (2001). *Advanced Encryption Standard (FIPS 197)*.
- NIST. (2013). *Digital Signature Standard (FIPS 186-4)*.
- Anderson, R. (2020). *Security Engineering* (3rd ed.). Wiley.

- `https://csrc.nist.gov` – NIST Computer Security Resource Center
- `https://www.cryptography.com` – Introduction resources

Thank You!

Questions & Discussion

"Cryptography is the essential building block of independence for organisations on the internet."
— Whitfield Diffie

CCY2001 – Introduction to Cybersecurity
Dr. Hany Elshall